

# Atividade em equipe

## Primeira tarefa do projeto em equipe

### Descrição da Linguagem

A linguagem proposta foi desenvolvida com o objetivo de ser simples, intuitiva e voltada para o ensino de programação. Diferente de linguagens tradicionais, ela utiliza palavras-chave em português, facilitando a compreensão por iniciantes.

Sua estrutura é organizada em blocos bem definidos, iniciando com a palavra reservada `projeto`, seguida da seção principal (`inicio`) e finalizando com `fimprojeto`.

A linguagem possui tipagem explícita, permitindo a declaração de variáveis dos tipos inteiro (`Int`), real (`RiR`), texto (`txt`) e lógico (`lgc`). As variáveis devem seguir regras de identificação, não podendo utilizar palavras reservadas.

Além disso, suporta operações matemáticas, relacionais e lógicas, possibilitando a construção de expressões complexas. Para interação com o usuário, a linguagem oferece comandos de entrada (`memo`) e saída (`mostre`).

Também inclui estruturas de controle, como condicionais (`se`, `senao`) e de repetição (`enquanto`, `repita`), permitindo controle do fluxo do programa. Por fim, a linguagem permite a definição de funções com parâmetros e retorno, promovendo reutilização de código e organização lógica dos programas. Tivemos como inspiração as linguagens C, Java e Python.

### Alfabeto

O alfabeto da linguagem é composto por letras maiúsculas e minúsculas, dígitos numéricos e caracteres especiais. Esses elementos formam a base para a construção de palavras, comandos e expressões dentro da linguagem, garantindo flexibilidade na escrita dos programas.

$$\Sigma = \{ a...z, A...Z, 0...9, / \% \wedge = < > ! \& | ( ) \{ \} [ ] ; , : . " ' \ + - \}$$

### Palavras reservadas

A linguagem deve possuir palavras reservadas fixas (não podem ser usadas como nome de variáveis). São em português para facilitar o entendimento. São elas: `projeto`, `inicio`, `fimprojeto`, `funcao`, `retorna`, `se`, `senao`, `enquanto`, `para`, `mostre`, `memo`, `int`, `real`, `txt`, `lgc`, `vrdd`, `flso`.

### Estrutura geral do programa

Todo programa na linguagem segue uma estrutura básica composta por um nome, um bloco de início e um bloco de fim. Essa organização ajuda a delimitar claramente onde o programa começa e termina, tornando o código mais organizado e fácil de entender.

```
projeto Nome_Projeto;  
inicio  
    //Comandos  
fimprojeto;
```

## Variáveis

As variáveis são utilizadas para armazenar dados durante a execução do programa. Elas seguem regras simples de nomenclatura e podem representar diferentes tipos de dados, como números inteiros, reais e textos.

Regras:

- Devem começar com letra;
- Podem conter números;
- Não podem ter espaços.

Os seus tipos são como segue:

- int: números inteiros;
- real: números reais;
- txt: sequência de caracteres;
- lgc: valores lógicos, verdade (vrdd) e falsidade (flso).

## Operações Matemáticas

A linguagem permite a realização de operações matemáticas básicas, como adição, subtração, multiplicação, exponenciação, raiz quadrada etc. Essas operações são essenciais para cálculos e manipulação de valores dentro dos programas. Os operadores são:

- + Adição (soma de valores)
- - Subtração (diferença entre valores)
- \* Multiplicação
- / Divisão
- % Resto da divisão (módulo)
- ^ Exponenciação

## Entrada e saída de dados

A linguagem possui comandos específicos para entrada e saída de dados. O comando de entrada permite que o usuário forneça informações ao programa, enquanto o comando de saída exibe resultados na tela, facilitando a interação com o usuário.

Comando de saída “*mostrar*”

O comando de saída *mostrar* tem o funcionamento inspirado no comando *printf*, presente nas linguagens C e Python.

*mostrar* (expressão)

Exemplo:

*mostrar* (“Olá Mundo!”);

Comando de entrada “*memo*”

O comando de entrada *memo* tem o funcionamento inspirado no comando *scanf* presente na linguagem C.

*memo* (expressão)

Exemplo:

int x;

mostre (“Digite um número inteiro: ”);

*memo* (x);

## Estruturas de decisão

As estruturas de decisão servem para que o programa “tome decisões”, ou seja, escolha o que fazer dependendo de uma situação. Na linguagem proposta, isso é feito com o comando “se”, que verifica uma condição. Se essa condição for verdadeira (vrdd), o programa executa um bloco de comandos; caso seja falsa (falso), ele pode executar outro bloco usando o “senao”. Um “se” pode ser aninhado em um “senao”.

Exemplo:

```
se (x > 0)
    mostre("O número é positivo");
senao
    mostre("O número não é positivo");
```

## Estruturas de repetição

As estruturas de repetição permitem que o programa execute um mesmo conjunto de comandos várias vezes, sem a necessidade de repetir código manualmente.

O comando “*enquanto*” funciona verificando uma condição antes de executar o bloco: enquanto essa condição for verdadeira, os comandos continuam sendo executados. Quando a condição se torna falsa, a repetição é encerrada. Já o comando “*repita*” executa primeiro o bloco de comandos e só depois verifica a condição. Isso garante que o código dentro do bloco seja executado pelo menos uma vez. O comando para é usado para repetir um bloco de comandos de forma controlada. Ele define, em uma única estrutura, a inicialização, a condição e a atualização da variável, executando o laço enquanto a condição for verdadeira.

Estrutura “*enquanto*”

```
enquanto (expressão)
    (bloco)
```

Exemplo:

```
int x;
x = 0;
enquanto (x < 5)
    mostre(x);
    x = x + 1;
```

Estrutura “*repita*”

```
int x;
x = 0;
repita
    mostre(x);
    x = x + 1;
enquanto (x < 5);
```

Estrutura “*para*”

```
int i;
para (i = 0; i < 5; i = i + 1)
    mostre(i);
```

## Gramáticas

<programa> → projeto id ; <bloco> fimprojeto

<bloco> → inicio <comandos>

<comandos> → <comando> <comandos> | ε

<comando> → <declaracao> | <atribuicao> | <saida> | <entrada> | <condicional> |  
<enquanto> | <repita> | <para> | <funcao>

<declaracao> → <tipo> id ;

<tipo> → int | real | txt | lgc

<atribuicao> → id = <expr> ;

<saida> → mostre ( <expr> ) ;

<entrada> → memo ( id ) ;

<condicional> → se ( <condicao> ) { <comandos> } | se ( <condicao> ) { <comandos> }  
senao { <comandos> }

<enquanto> → enquanto ( <condicao> ) { <comandos> }

<repita> → repita { <comandos> } enquanto ( <condicao> ) ;

<para> → para ( id = <expr> ; <condicao> ; id = <expr> ) { <comandos> }

<expr> → <expr> + <termo> | <expr> - <termo> | <termo>

<termo> → <termo> \* <fator> | <termo> / <fator> | <termo> % <fator> | <fator>

<fator> → ( <expr> ) | id | numero | string | vrdd | flso

<condicao> → <expr> <op\_rel> <expr>

<op\_rel> → = | != | < | > | <= | >=

<funcao> → funcao id ( <parametros> ) : <tipo> <bloco> <retorno> fimprojeto

<parametros> → <lista\_param> | ε

<lista\_param> → <tipo> id | <tipo> id , <lista\_param>

<retorno> → retorna <expr> ;